

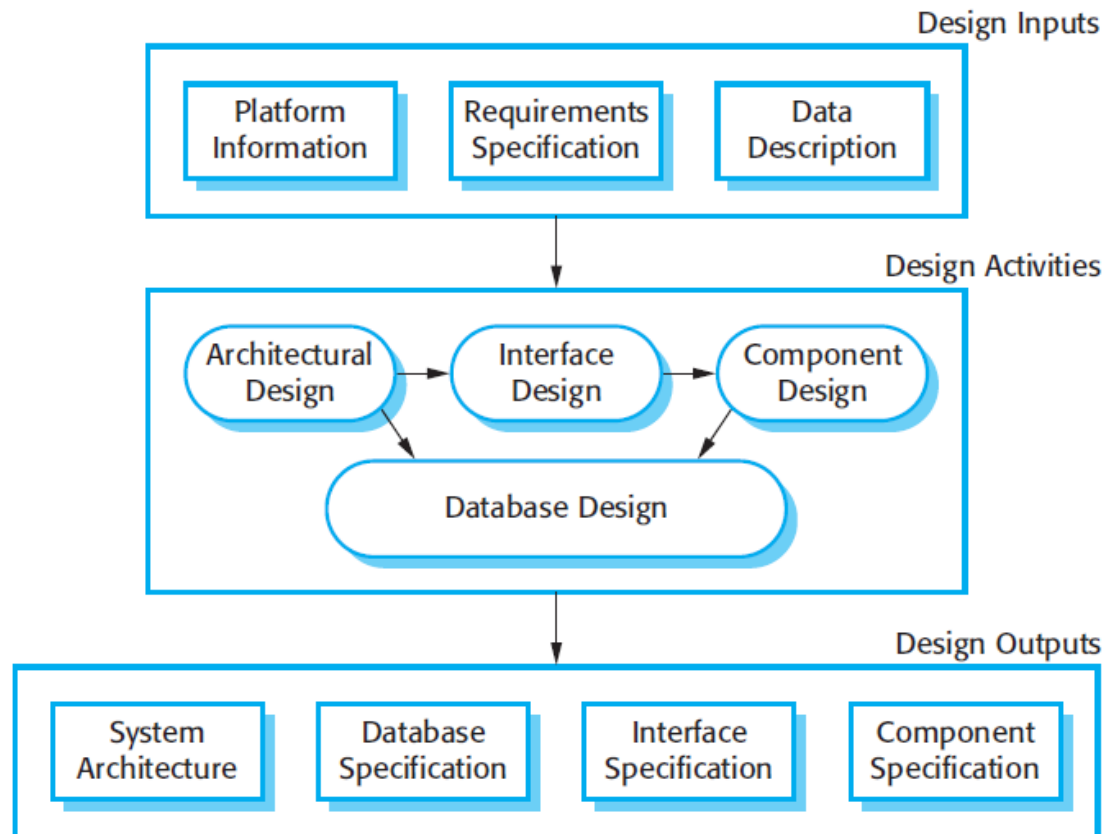
Công nghệ phần mềm

Thiết kế chi tiết

Bức tranh lớn

Trong giai đoạn thiết kế, nhà phát triển có thể chia làm các hoạt động thiết kế tách biệt

- thiết kế kiến trúc mức cao (đã học)
- thiết kế giao diện (đã học)
- **thiết kế thành phần** (bài học này) – Component Design
- **thiết kế CSDL** (bài học này)



THIẾT KẾ THÀNH PHẦN

Ôn lại

- Thiết kế kiến trúc mức cao (high-level design)
 - Giúp xác định các **thành phần chương trình** (program components) chính để xây dựng phần mềm, và cấu trúc dữ liệu
 - Mối quan hệ giữa các thành phần chương trình
 - Được tạo ra bởi chuyên gia
- Thành phần chương trình

“... a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.”
[OMG03a]

Thiết kế mức thành phần

- Vấn đề: thiết kế tổng quan vẫn ở mức **chung chung**, chưa đi sâu vào chi tiết. Trong khi đó, lập trình phải được thực hiện dựa trên bản thiết kế chi tiết hơn
 - > thực hiện thiết kế mức thành phần

- Ai thực hiện?

Dev là đủ

Góc nhìn Thành Phần

- Ta cần có các góc nhìn (view) khác nhau để hiểu về thành phần
 - Góc nhìn hướng đối tượng (Object-Oriented View)
 - Góc nhìn truyền thống (Traditional View) – đọc thêm
 - Góc nhìn quy trình (Process-Related View) – đọc thêm

Góc nhìn hướng đối tượng

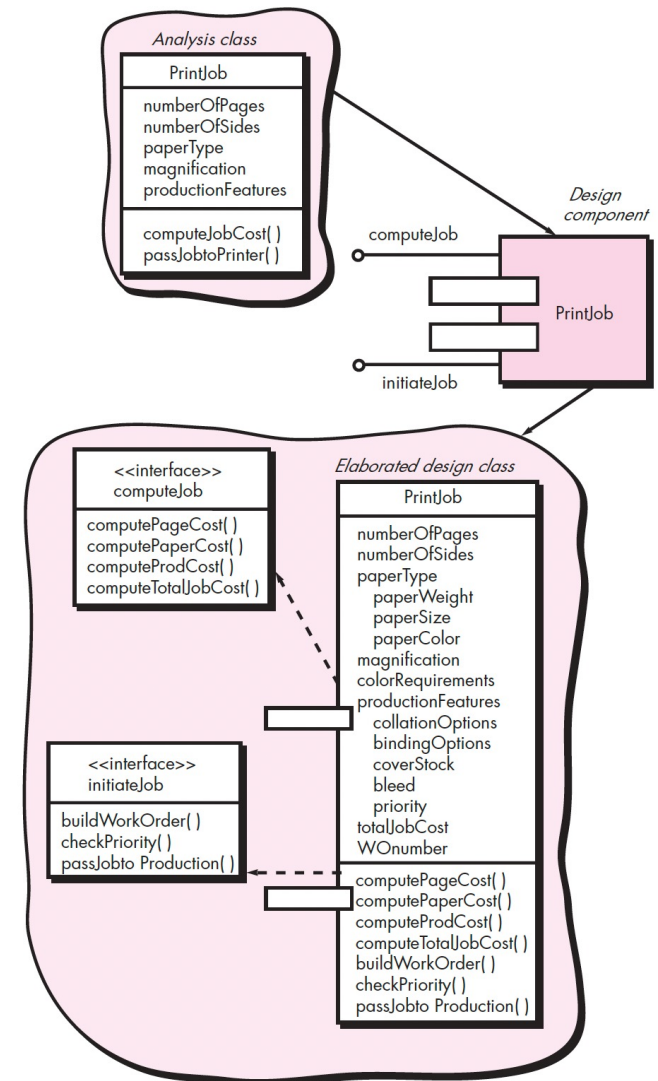
- Một thành phần gồm nhiều lớp (class) có liên quan đến nhau và có tương tác với nhau
- Khi biểu diễn từng lớp, thuộc tính và phương thức cốt lõi sẽ được thể hiện

Góc nhìn hướng đối tượng

- Xác định mức độ truy xuất của các thuộc tính và phương thức
 - Private: chỉ trong lớp đối tượng
 - Package: trong cùng gói (package)
 - Protected: lớp con và cùng gói
 - Public: mọi nơi

Góc nhìn hướng đối tượng

- Ví dụ: Cửa hàng in ấn
 - Trong quá trình thu thập, phân tích, và đặc tả yêu cầu, một component tên **PrintJob** được tạo. Mô tả thô sơ của component này được thể hiện ở Analysis class.
 - Trong quá trình thiết kế mức cao, chuyên gia thiết kế
 - component **PrintJob** có tương tác vs component khác thế nào (hai giao diện tên **computeJob** – tính giá tiền in ấn và **initiateJob** – khởi tạo một phiên in ấn)
 - Thuộc tính cơ bản và phương thức cơ bản
 - Sau đó, dev áp dụng thiết kế mức thành phần (dùng góc nhìn hướng đối tượng)
 - **PrintJob** được phân rã chi tiết hơn để phục vụ cho lập trình (elaborated design class)



Nguyên lý thiết kế HĐT (1/2)

- SRP (The Single Responsibility Principle)

Một lớp đối tượng chỉ có một nhiệm vụ duy nhất

- OCP (The Open Closed Principle)

Lớp đối tượng phải thiết kế sao cho dễ dàng mở rộng mà không cần phải thay đổi

- LSP (The Liskov Substitution Principle)

Các lớp con phải thay thế được cho các lớp cha

Nguyên lý thiết kế HĐT (2/2)

- ISP (The Interface Segregation Principle)

Phân tách các giao diện sao cho client không phải phụ thuộc vào giao diện nó không sử dụng

- DIP (The Dependency Inversion Principle)

Phụ thuộc vào lớp trừu tượng thay vì lớp cụ thể

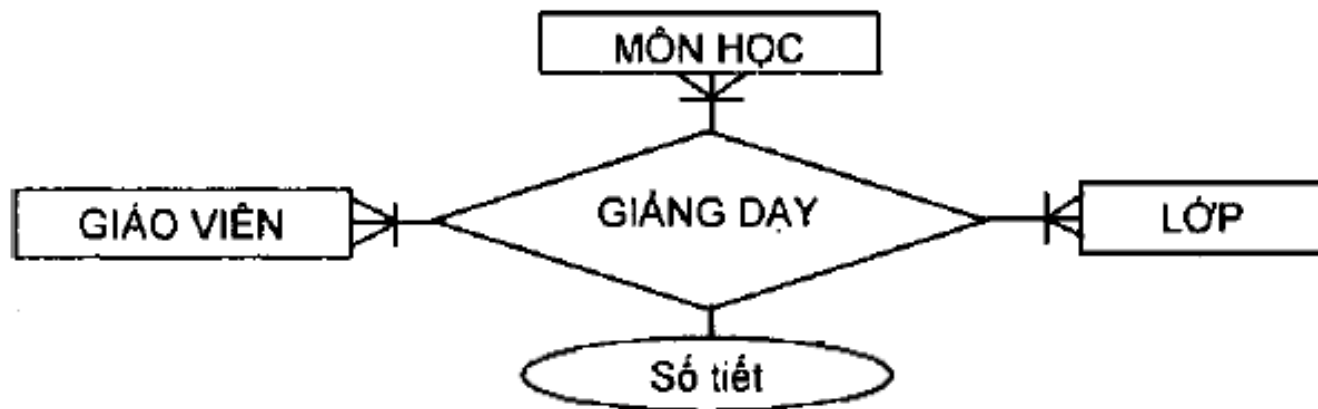
THIẾT KẾ DỮ LIỆU

Mô hình thực thể – quan hệ

- Là dạng tiền sơ khai để tạo ra CSDL hoàn chỉnh
- Các bước chính xây dựng ERM
 - B1: Xác định các thực thể, các thuộc tính
 - B2: Xác định các mối quan hệ và thuộc tính của mỗi quan hệ
 - B3: Vẽ biểu đồ ERM (nhưng ERM có thể chưa tối ưu)
 - B4: Chuẩn hóa ERM

Mô hình thực thể - quan hệ

- Thực thể *Giáo viên*, *Môn học*, và *Lớp*
- Quan hệ giữa các thực thể này là *Giảng dạy*
- *Số tiết* là thuộc tính của quan hệ



Ánh xạ lớp đối tượng vào CSDL

- Lớp đối tượng được ánh xạ vào một bảng (thường sẽ có cùng tên)
- Các thuộc tính với kiểu dữ liệu đơn giản (string, nguyên thủy) sẽ ánh xạ vào cột
- Quan hệ One-to-one được thực hiện bằng cách thêm khóa từ một bảng vào bảng còn lại
- Quan hệ One-to-many: thêm khóa của bảng phía “one” vào bảng “many”
- Quan hệ Many-to-many: tạo bảng mới chứa khóa của cả hai bảng

**Any
questions**



VectorStock®

[VectorStock.com/7478164](https://www.vectorstock.com/7478164)